

Diseño del Gemelo Digital de Iones: Adaptación de la Arquitectura VALeR / CBOL-Tuner

Documento de Arquitectura y Metodología

1. El Problema y la Representación de los Datos

En los trabajos de Rautela et al., el estado del sistema X está compuesto por imágenes de alta resolución (proyecciones de fase 6D). En nuestro beamline de SIMION, el estado del sistema en el detector puede representarse de forma más compacta, pero queremos conservar la riqueza espacial del haz para optimizar mejor.

Recomendación de Datos (X): En lugar de modelar solo la "transmisión" (un escalar), te sugiero modelar un **histograma 2D de baja resolución** de las llegadas de los iones al detector (por ejemplo, una cuadrícula de 20×20 bins), aplanado a un vector de 400 dimensiones.

- **¿Por qué?** Porque si modelas la distribución espacial, tu modelo entenderá la *forma* del haz. Optimizar para tener un haz concentrado en el centro del detector es un problema mucho más rico y útil que solo maximizar un conteo. Si sumas todos los bins del histograma, obtienes automáticamente la transmisión.

Los voltajes de control serán nuestro vector de acciones: $y \in \mathbb{R}^8$.

2. Flujo de la Arquitectura del Gemelo Digital

Tu Gemelo Digital estará compuesto por **tres sub-redes** que se entrenan en fases. Esto te garantiza robustez, diferenciabilidad analítica (Rúbrica D), y control de extrapolación (Rúbrica E).

Fase 1: El Autoencoder Variacional (VAE) - Compresión del Estado

Entrenamos un VAE simple con capas densas (MLP) para comprimir el histograma del haz X en un espacio latente de muy baja dimensión z (e.g., $z \in \mathbb{R}^2$ o $\mathbb{R}^3$).

- **Encoder:** $q_\phi(z|X) \rightarrow \mu_z, \sigma_z$
- **Decoder:** $p_\theta(X|z) \rightarrow \hat{X}$
- **Loss:** Error de reconstrucción (MSE) + Divergencia KL.

Ventaja: El VAE obliga a que las distribuciones de haz "físicamente posibles" vivan en una variedad (manifold) continua y suave en el espacio latente z .

Fase 2: El Emulador Forward (El Gemelo en sí)

Para la hackathon, necesitas un modelo que reciba voltajes y y prediga el resultado X . En lugar de mapear $y \rightarrow X$ directamente (lo cual es ruidoso y propenso a predecir ruido no físico), entrenamos un regresor que mapee los voltajes al espacio latente:

- **Forward Regressor (DNN o Gaussian Process):** $f_\psi(y) \rightarrow \hat{z}$

Flujo del Emulador: $y \xrightarrow{f_\psi} \hat{z} \xrightarrow{Decoder} \hat{X}$

- *Nota para la rúbrica (D):* Todo este flujo (DNN + Decoder) se construye en PyTorch o TensorFlow y es trivialmente diferenciable usando backpropagation (`.backward()`). Obtendrás el gradiente $\nabla_y X$ al instante.

Fase 3: El Estimador Inverso (Para la Consigna Inversa)

Para la segunda tarea de control de la hackathon (Consigna Inversa), necesitamos ir del resultado deseado a los voltajes. Como sugiere el *CBOL-Tuner*, entrenamos una pequeña red densa:

- **Estimator (DNN):** $g_{\omega}(z) \rightarrow \hat{y}$

Si te dan un perfil de haz objetivo X_{target} , lo pasas por el Encoder para obtener z_{target} , y luego por el Estimator para obtener los voltajes $y = g_{\omega}(z_{target})$. ¡Tarea resuelta en un par de milisegundos!

3. El Reemplazo de ResNet50: "Latent Density Pruning"

En el artículo de Rautela, el optimizador Bayesiano explora el espacio latente z , pero a veces encuentra puntos z que, al decodificarse, dan distribuciones de haz imposibles (alucinaciones del VAE en zonas vacías del espacio latente). Usaron ResNet50 para clasificar y rechazar (podar) estas imágenes irreales.

Como tú no tienes imágenes ni un clasificador pre-entrenado, **¿cómo aplicamos el "Classifier-Pruning" para asegurar la Admisibilidad (Rúbrica G)?**

La solución rigurosa: Modelado de Densidad en el Espacio Latente (Latent Density Pruning). Dado que tu espacio latente z es de dimensión muy baja (e.g., 2 o 3), puedes estimar explícitamente la densidad de probabilidad de tus datos de entrenamiento en ese espacio.

1. **Entrenamiento del "Podador":** Una vez entrenado el VAE, pasa todos tus datos reales de SIMION por el Encoder para obtener una nube de puntos $\{z_1, z_2, \dots, z_N\}$.
2. **Estimación de Densidad:** Ajusta un **Gaussian Mixture Model (GMM)** o un **Kernel Density Estimator (KDE)** sobre esta nube de puntos. Alternativamente, puedes usar un **Isolation Forest**.
3. **El Criterio de Poda (Pruning):** Cuando tu Optimizador Bayesiano evalúe un nuevo candidato z^* , primero evalúas su probabilidad $P(z^*)$ en tu GMM/KDE.
 - Si $P(z^*) < \text{Umbral}$ (es decir, el optimizador se salió de la zona donde existe física real), rechazas ese punto y le devuelves a Optuna un valor de pérdida infinito (o muy malo).

Impacto en la Hackathon: Esto garantiza que tu modelo NUNCA sugiera voltajes que produzcan resultados no físicos, blindando tu puntaje en el factor de admisibilidad G .

4. Tarea de Dirección (Control y Optimización)

Para encontrar los mejores voltajes (e.g., maximizar transmisión y enfocar el haz), aplicamos la filosofía del *CBOL-Tuner*. En lugar de usar Optuna para buscar ciegamente en el vasto espacio de $\mathbb{R}^8$ (los 8 voltajes), optimizamos en el espacio latente.

El Algoritmo de Búsqueda:

1. Definimos la función objetivo en el espacio latente: $J(z) = \text{Score}(\text{Decoder}(z))$. Por ejemplo, qué tan centrado e intenso es el histograma resultante.
2. Usamos Optimización Bayesiana (Gaussian Process + Expected Improvement) para buscar el z^* óptimo en $\mathbb{R}^2$ o $\mathbb{R}^3$.
3. **C-BO (Classifier-Pruned BO):** Durante la búsqueda, cualquier z propuesto por el Proceso Gaussiano se pasa por tu Filtro de Densidad (KDE/GMM). Si es una anomalía, se poda.
4. Una vez encontrado el z^* óptimo (que sabemos que es físicamente válido), usamos tu *Estimator Inverso* de la Fase 3: $y^* = g_{\omega}(z^*)$ para obtener los 8 voltajes.

Esta búsqueda en 2D/3D latente es órdenes de magnitud más rápida y estable que buscar en el espacio de 8 dimensiones original de voltajes.

5. Recolección de Datos (Active Learning)

Para maximizar el término F (Eficiencia de datos) y E (Extrapolación/Incertidumbre), no recolectes tus puntos aleatoriamente.

1. **Generación de semilla:** Usa un *Latin Hypercube Sampling* (LHS) en los voltajes para generar unos 50-100 puntos iniciales con SIMION.
2. **Aprendizaje Activo:** Entrena una primera versión de tu modelo Forward. Usa un Ensamble de DNNs o un Proceso Gaussiano (GP) para mapear $y \rightarrow z$.
3. Calcula la incertidumbre (varianza) de las predicciones de tu modelo en puntos no explorados.
4. Llama a SIMION *solo* en las combinaciones de voltajes donde tu modelo tiene alta incertidumbre y que tengan potencial de ser buenas operaciones.
5. Actualiza el modelo. Con esto, tu modelo sabrá exactamente cuándo decir "no sé" (fundamental para un alto puntaje en la Rúbrica E) porque estará calibrado basándose en su varianza predictiva.